SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:50**

Mode: In Class **Hours : 60 mts**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I Kola Amrutha Sandhya (192524270) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	**/ 20**	**/ 25**	**/ 20**	**/ 15**	**/ 100**

Annexure A

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression P = (a * b) + (b * c) * 2 through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. a) b) c) d) e)

3	i)Find the number of tokens in the following C statement is
	main( )
	{
	int a=10;
	char b =”abc”;
	in t c = 30;
	ch ar d =“xyz”;
	in /* comment */ t m = 40.5;
	}
	ii)Identify the lexical errors in the following C statement is
	void main()
	{
	int x=10, y=20;
	char * a;
	a= &x;
	x= 1xab;
	}

4	**Find a regular expression corresponding to each of the following subsets of :** i) The language of all strings that have an even number of 0’s. ii) The language of all strings that contain at least one 1. iii) The language of all strings in which both the number of 0’s and the number of 1’s are odd. iv) The language of all strings that start with 1 and end with 0. v) The language of all strings that do not contain consecutive 1’s.
	Consider a lexical analyzer for a simplified programming language. The language consists

of the following tokens:

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, *, /, =, ==, <, >
3. Separators: (,), {, }, ;, ,
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string:

int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }

Questions:

a) How many tokens are there in the given input string?

b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).

c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

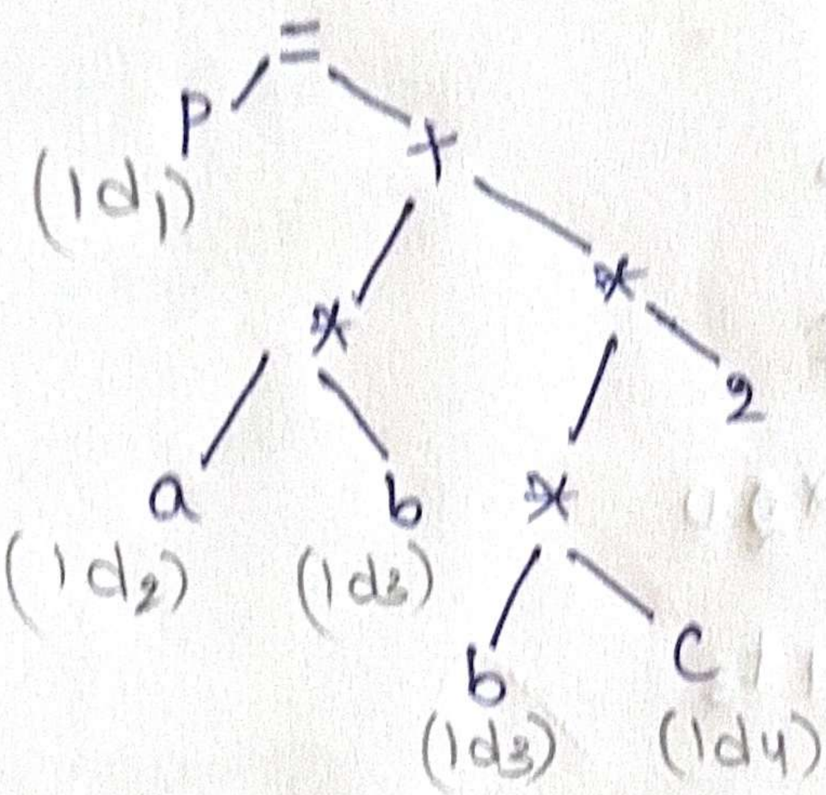
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

given $p = (a * b) + (b * c) * 2$

a) Lexical Analysis

$id_1 = (id_2 * id_3) + (id_3 * id_4) * 2$

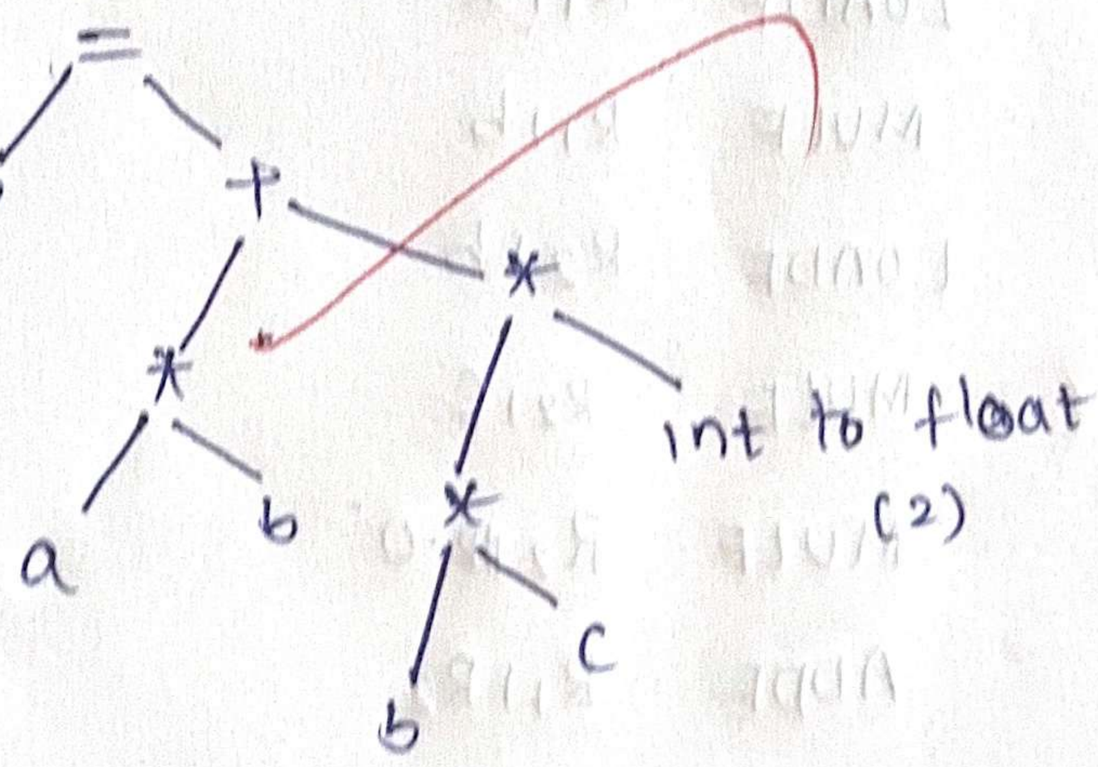
b) Syntax Analysis



90/100

2M/05/10/24.

Semantic Analysis



c) Intermediate code Generator

$t_1 = a * b$

$t_2 = b * c$

$t_3 = t_2 * t_2$

$t_4 = t_1 + t_3$

$p = t_4$

d) code optimization

$t_1 = a * b$

$t_2 = b * c$

$t_3 = t^2 * 2.0$

$P = t_1 + t_3$

e) Code Generation

LOADF R_1 a

MULF R_1 b

LOADF R_2 b

MULF R_2 C

MULF R_2 #2.0

ADDF R_1 R_2

&TRF p R_1

$(\varepsilon | a\, a^*)^*$

ε, a, aa, aaa, aaaa

b) $(a^* | b^*)^*\, abb$

abb, aabb, babb, aaabb, ababb

c) $(a^* b^*)^*\, aba\, (a^* + b^*)^*$

aba, aaba, abaa, aabaa, ababa

d) $(a | ab)^*$

ε, a, ab, aa, aab

e) $(abab)^* + (a^* b^*)^*$

ε, a, b, ab, abab

3) (i) Find the number of TOKENS

Given

```
main()
{
 int a=10;
 char b="abc";
 int c = 30;
 char d = "xy2";
 in/* comment */t m=40.5;
}
```

Tokens:-

main, (,), {, int, a, =, 10, ;, char, b, =, "abc", ;, in, t, c, =, 30, ;, ch, ar, d, =, "xyz", ;, in, t, m, =, 40.5, ;, }

Total = 33 tokens

ii) Lexical errors Identification

Given

```
void main()
{
  int x=10, y=20;
    char *a;
    a= &x;
    x=1xab;
}
```

Lexical Error

x=1xab;

4) Regular Expressions

(i) String having an even number of 0's

1*(01*01*)*

iii) strings containing at least one 1

$0^*10^*(0^*10^*)^*$

$(0+1)^*1(0+1)^*$

iii) strings in which both number of 0's and 1's are odd

$0(00)^*1(11)^* + 1(11)^*0(00)^*$

iv) strings that start with 1 and end with 0

$1(0+1)^*0$

v) strings that do not, contain consective 1's

$(0+10)^*(\varepsilon+1)$

5) Lexical Analyzer

Given input

```
int var123 = 12345;
float var456 = 123.45;
if (var123 == var456) {return;}
```

a) Number of tokens

int , var123 , = , 12345 , ; , float , var456 , = , 123.45 , ; ,
if , (, var123 , = = , var456 ,) , { , return , ; , }

Total = 20

b) Tokens categorized by type

Keywords

int
float
if
return

Identifiers

var123
var456

var 123
var 456

Operators

=
==
=

Separators

;
;
(
)
{
;
}

Integer literals

12345

Floating point literals

123.45

c) FSM State Transitions

Token	Transitions
int	3
var123	6
=	1
12345	5
;	1
float	5
var456	6
=	1
123.45	6
;	1
if	2
(	1
var123	6
==	2
var456	6
)	1
{	1

return	=	6
;		1
}		1

Total = 62